IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 15 October 2025, accepted 17 November 2025, date of publication 1 December 2025, date of current version 5 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3638925

RESEARCH ARTICLE

Lightweight FPGA-Based Number-Theoretic Transform for ML-KEM in Post-Quantum Cryptography

MOHAMMED O. ALSHOMRANY[1], MUHAMMAD AL-HASHIMI[1], FAWAZ ALSOLAMI[1], MOSTAFA SALEH[1], OSAMA ABULNAJA[1], TURKI F. AL-SOMANI[2], AND ABDULLAH ALTUWAIJRI[3]

[1]Department of Computer Science, Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah 25732, Saudi Arabia
[2]Department of Computer Engineering, Umm Al-Qura University, Mecca 21955, Saudi Arabia
[3]National Company of Telecommunications and Information Security (NTIS), Riyadh 13315, Saudi Arabia

Corresponding author: Mohammed O. Alshomrany (malshomrany0007@stu.kau.edu.sa)

This work was supported by the King Abdulaziz University (KAU) Endowment (WAQF) at King Abdulaziz University, Jeddah, Saudi Arabia.

ABSTRACT This study introduces a lightweight, energy-efficient Number Theoretic Transform (NTT) implementation tailored for the ML-KEM scheme in post-quantum cryptography. Unlike prior efforts focusing mainly on multipliers and modular reduction, the proposed design emphasizes the address generator, a critical yet often overlooked component. A compact counter-based mechanism integrated with XOR logic and multiplexing eliminates large look-up tables, reduces redundant operations, and ensures efficient memory access for both NTT and INTT. The unified butterfly unit and a LUT-based modular reduction circuit enable multi-cycle modular operations without requiring digital signal processing (DSP) or block RAM (BRAM) resources. The complete architecture occupies only 147 slices and 557 LUTs, achieving very low power consumption. Implemented on a Virtex-7 platform, the design demonstrates up to 88.8% reduction in area, 22% decrease in dynamic power, and a 17.57× improvement in A^2T efficiency compared with state-of-the-art implementations. These results highlight the significance of optimizing address generation to achieve compact, power-aware accelerators. The architecture's elimination of memory-intensive components and reliance on LUT-based logic make it highly portable and suitable for resource-constrained devices.

INDEX TERMS CRYSTALS-Kyber, FPGA design, Internet of Things, lightweight post-quantum cryptography, ML-KEM, number-theoretic transform, resource-constrained devices.

I. INTRODUCTION

As the Internet of Things (IoT) continues its rapid expansion, becoming a cornerstone of modern infrastructure, IoT devices have advanced to feature longer lifespans, smaller form factors, and greater intelligence [1]. Consequently, securing digital communications is paramount to safeguarding data integrity and confidentiality. Cryptographic mechanisms protect sensitive data across various domains, from online banking to home automation.

Post-quantum cryptography schemes such as CRYSTALS–Kyber rely heavily on the number-theoretic transform (NTT) and its inverse (INTT) to perform efficient polynomial multiplication. However, these operations constitute most of the computational workload and introduce significant area and power overheads, particularly in address-generation logic and butterfly computation units. In resource-constrained IoT devices, such inefficiencies become bottlenecks, limiting the practical deployment of post-quantum cryptography.

This study addresses these challenges by introducing a lightweight FPGA-optimized NTT/INTT architecture that concurrently reduces area, power, and resource utilization. The design integrates a record-low-resource address generator with an optimized unified butterfly module, achieving efficiency gains without compromising throughput.

The associate editor coordinating the review of this manuscript and approving it for publication was SK Hafizul Islam.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

A. NEED FOR POST-QUANTUM CRYPTOGRAPHY (PQC)

Cryptographic techniques encompass both symmetric and asymmetric encryption. Symmetric schemes (e.g., AES and ChaCha20) utilize a shared key, requiring secure key distribution [2]. By contrast, asymmetric schemes (e.g., Rivest–Shamir–Adleman [RSA] and elliptic curve digital signature algorithm [ECDSA]) employ a public–private key pair, which eliminates the need for secure key sharing [3].

Although symmetric encryption safeguards user privacy, asymmetric schemes are essential for digital signatures and key-exchange protocols. Currently, public-key cryptography secures over 60% of Internet connections through protocols such as Transport Layer Security (TLS), which establish encrypted communication channels [4], [5]. TLS utilizes digital certificates issued by trusted authorities to authenticate and confirm the legitimate association between a public key and its corresponding identity. ECDSA and RSA [6] are the most widely used public-key encryption schemes for digital certificates.

The security of these schemes hinges on the computational difficulty of their underlying mathematical problems without access to the private key. Specifically, RSA depends on the challenge of integer factorization, whereas ECDSA relies on the elliptic curve–discrete logarithm (ECDL) problem. Currently, classical computing renders the cryptanalysis of these schemes impractical owing to prohibitively long runtimes. However, quantum computing introduces fundamental vulnerabilities through algorithms that can solve these problems in polynomial time.

Shor's algorithm poses a significant threat to the security foundations of RSA and ECDSA [7], as it efficiently factors integers and solves elliptic curve–discrete logarithm problems, thereby undermining the hardness assumptions of these schemes and making them susceptible to quantum attacks. Although large-scale quantum computers capable of executing Shor's algorithm are not yet available, rapid advancements by IBM and Google underscore the urgency of developing quantum-resistant cryptographic solutions [8], [9].

To address the vulnerabilities posed by quantum computing, researchers are actively exploring PQC, which is designed to withstand quantum attacks. The National Institute of Standards and Technology (NIST) has led the standardization of PQC and selected CRYSTALS-Kyber, standardized as ML-KEM, as the preferred key encapsulation mechanism (KEM) because of its robust security guarantees and efficient implementation in key generation, encapsulation, and decapsulation [1]. ML-KEM is based on the module learning with errors (MLWE) problem, deriving its security from the computational hardness assumptions inherent in module lattices [10].

As PQC moves toward widespread standardization, efficient hardware implementations become crucial for resource-constrained IoT devices. Although lattice-based cryptography ensures security through the computational difficulty of module lattices, it also introduces significant computational complexity, especially in polynomial arithmetic. Optimized hardware architectures must balance security with stringent constraints on area, power, and latency. This balance is essential for the practical adoption of PQC in IoT devices.

B. COST-COMPLEXITY TRADE-OFF IN CRYPTOGRAPHIC HARDWARE

Cryptographic hardware implementations must consider energy consumption, area efficiency, and execution time based on application requirements. For IoT sensor nodes and RFID tags, minimizing area often remains the primary objective. Conversely, data centers and secure network gateways may prioritize reducing latency even at the cost of higher energy consumption [11].

A cost–complexity metric has been proposed to quantify hardware trade-offs, where A represents area and T denotes execution time. The metric is defined as follows:

$$Cost\ Complexity = Ax \times Ty \tag{1}$$

where x and $y \in \{0, 1, 2\}$ represent the relative priorities of area (A) and time (T), respectively. A value of zero signifies no importance, whereas two indicates high significance [11]. For devices with limited processing capacity, prioritizing area efficiency over performance is crucial. A commonly adopted specialization of this metric is

$$Cost\ Complexity = A^2T. \tag{2}$$

In resource-constrained systems, the physical size of components is critical. Therefore, optimizing for area efficiency has a more substantial impact on functionality than optimizing performance [12], [13]. Energy consumption is influenced by the area footprint, operating frequency, and memory usage; for instance, larger models or data structures lead to more memory-access events and storage operations, thereby increasing energy demands. Execution time directly affects energy consumption, as longer runtimes result in higher overall usage [14], [15]. By optimizing cryptographic implementations for area constraints, devices can achieve secure and efficient PQC execution within hardware limitations.

C. KEY CONTRIBUTIONS

This study presents a lightweight FPGA implementation of the NTT, optimized for ML-KEM in resource-constrained IoT environments. The key contributions are listed below:

1) LIGHTWEIGHT ADDRESS GENERATOR WITH VERY LOW RESOURCE USAGE

This module contributes only 2 mW to dynamic power consumption (approximately 25% of the total NTT module power), compared to 11–23 mW reported in other studies.

2) HARDWARE-OPTIMIZED UNIFIED BUTTERFLY MODULE

The unified butterfly architecture removes dependence on DSP resources; it maintains peak throughput, thereby

balancing minimal resource usage with maximum processing speed. The module consumes only 8 mW of dynamic power, further reducing area and power usage in conjunction with the proposed address generator.

3) HARDWARE-OPTIMIZED FULL NTT/INTT ARCHITECTURE

This compact, BRAM-free and DSP-free architecture handles polynomial coefficient access using only 49 look-up tables (LUTs) and 28 FFs, achieving a 90%–94% reduction compared to recent state-of-the-art designs. When integrated with the optimized address generator and butterfly module, the NTT/INTT implementation on a Xilinx Artix-7 FPGA delivered competitive throughput per slice and significantly enhanced A^2T (Area$^2\times$ Time) efficiency over existing solutions.

4) VALIDATED SYNTHESIS AND OPERATIONAL PERFORMANCE

The architecture was validated at operational frequencies up to 100 MHz through post-synthesis and post-implementation timing analyses. Static and dynamic power distributions were detailed for all major modules, facilitating benchmarking against other studies reporting power metrics.

By targeting both the addressing logic and computational core of the NTT, this study addressed the primary area and power constraints in PQC accelerators for IoT devices, making CRYSTALS-Kyber polynomial multiplication more practical in the post-quantum era.

The remainder of this paper is organized as follows. Section II introduces the study background, Section III surveys related work, Section IV presents the proposed methodology, Section V details experimental results and discussion, and Section VI concludes with a look at future research directions.

II. BACKGROUND

ML-KEM is a cryptographic scheme offering robust security assurances, high performance, and relatively compact key and ciphertext sizes. The ML-KEM cryptographic mechanism allows two parties to derive a shared secret key without direct transmission, thereby minimizing security exposure and catering to resource-constrained devices through its efficiency and robust security guarantees [10]. Its security is based on the MLWE problem, which remains intractable even for quantum computers [16].

ML-KEM employs a two-phase construction to achieve an indistinguishable under chosen-ciphertext attack (IND-CCA) secure KEM. First, ML-KEM constructs a public-key encryption (PKE) scheme with IND-CPA security from the MLWE problem and subsequently converts it into a KEM using the Fujisaki–Okamoto (FO) transform [17], [18], thereby attaining resistance against chosen-ciphertext attacks.

ML-KEM offers three parameter sets—ML-KEM-512, -768, and -1024—aligned with NIST security levels 1, 3, and 5, respectively. These parameters balance security and efficiency, which enables flexible adoption based on application requirements [1]. Among these, ML-KEM-768 provides an optimal balance between security and computational efficiency.

Furthermore, polynomial multiplication in the ring Zq[x]/(xn+1) is one of the most computationally intensive operations, affecting performance and resource consumption, particularly in constrained IoT nodes. To mitigate this bottleneck, ML-KEM utilizes the NTT, enhancing polynomial multiplication efficiency and lowering computational complexity. However, the limited processing power and memory of IoT devices make deploying a lightweight yet secure cryptographic scheme a formidable challenge [19].

A. NTT IN ML-KEM

Polynomial multiplication is a fundamental operation in various PQC algorithms. Traditional methods, such as the schoolbook algorithm [20], [21], Karatsuba [22], [23], and Toom–Cook [24], [25], incur significant computational overhead, making them unsuitable for resource-constrained devices. To address this challenge, ML-KEM utilizes NTT, thereby enhancing efficiency. Although ML-KEM provides robust security, deploying it on IoT and embedded platforms remains challenging owing to limited resources. IoT devices typically have restricted RAM and ROM and operate with limited processing capabilities. Additionally, cryptographic processes must minimize energy consumption to prolong battery life.

ML-KEM exploits NTT to expedite polynomial multiplication and reduce the time complexity from O(n^2) to O(nlogn), thereby enhancing the computational efficiency. In contrast to Fast Fourier Transform (FFT), which relies on complex-domain floating-point arithmetic and is prone to precision errors, NTT operates solely with integer arithmetic, making it ideal for cryptographic applications. Moreover, NTT minimizes memory usage through in-place computation and facilitates parallelization via structured butterfly operations, which enable highly efficient FPGA implementations [19].

NTT is an integer-based variant of Discrete Fourier Transform (DFT) optimized for modular arithmetic. It substitutes the complex roots of unity in FFT with integer roots, ensuring exact modular computations without rounding errors [26]. This feature makes NTT particularly well-suited for lattice-based cryptography, where arithmetic precision is critical.

Beyond complete NTT/INTT implementations, numerous studies have explored subcomponents that directly impact NTT performance. Specifically, resource-efficient modular multipliers [27], [28] such as "*Efficient Reconfigurable Modular Multipliers for Post-Quantum Digital Signatures*" [29] and "*Efficient Soft Core Multiplier for Post-Quantum Digital Signatures*" [30] provide optimized LUT and carry-chain-based architectures tailored for post-quantum signature schemes. Although these designs do not implement complete NTT pipelines, their multiplier architectures can be integrated within NTT/INTT stages to minimize FPGA resource consumption and boost throughput in PQC accelerators.

ML-KEM operates within the structured polynomial ring R_q, where $q = 3329$ is a prime number and $n = 256$ signifies the polynomial degree. The transformations between coefficient representation and their NTT-domain counterparts are executed as follows:

$$\begin{aligned} R_x &= Z_x(X)/(X^n + 1), \\ \hat{f} &: = NTT(f), \\ \hat{f} \times R_x g &= NTT^1(f \times T_x \hat{g}), \end{aligned} \tag{3}$$

where $f \in Rq$ is converted to $f \in Tq$. The inverse transformation (NTT^{-1}) reverts to the coefficient form as expressed in Eq. (3). These transformations enable polynomial multiplication through straightforward point-wise multiplication in the NTT domain, thereby lowering computational overhead.

III. RELATED WORK

Numerous hardware architectures have been developed to optimize NTT operations, emphasizing area–performance trade-offs in FPGA implementations. We classify prior work into three main categories: (1) butterfly unit designs, (2) pipelined and memory-efficient architectures, and (3) modular reduction and hardware optimization techniques. Each contribution is evaluated based on its effectiveness in mitigating computational bottlenecks in ML-KEM implementations.

A. BUTTERFLY UNIT DESIGNS FOR EFFICIENT NTT COMPUTATION

The butterfly unit is a fundamental computational component in NTT operations, directly influencing performance, resource usage, and latency. Various studies have proposed designs to enhance butterfly unit efficiency.

Kieu-Do-Nguyen et al. [19] developed a compact, low-latency FPGA-based NTT accelerator for ML-KEM. The design featured a fully pipelined, high-frequency butterfly unit, achieving 417 MHz on an Artix-7 FPGA with minimal hardware resources. It introduced two butterfly variants: (1) a multiplication-free scheme utilizing LUTs for modular multiplication to eliminate DSPs, and (2) a quarter-square method requiring 1,024 dual-port ROM entries. The design optimized the logic area through algebraic simplification.

Similarly, Bisheh-Niasar et al. [27] reported a reconfigurable butterfly architecture for NTT and INTT, employing the Gentleman–Sande (GS) algorithm for INTT and Cooley–Tukey (CT) for forward NTT. Yahya Hummdi et al. [28] introduced the Unif-NTT accelerator, a platform-neutral unified butterfly unit that integrates CT and GS within a single module. Furthermore, the unified module supports dynamic switching between CT and GS operations, enabling flexible deployment across various hardware platforms. Implemented on FPGA and ASIC, this architecture significantly reduces computational overhead and enhances efficiency.

Furthermore, Xing and Li [31] present a unified butterfly pair supporting both decimation in frequency NTT and decimation in time INTT with specialized control codes and functional reuse across NTT, INTT, and point-wise multiplication operations by reusing idle adders/subtractors inside the butterfly units, thus saving hardware resources.

These studies demonstrate the effectiveness of optimized butterfly units in improving performance. Nonetheless, existing solutions continue to face challenges in ultralow-power IoT environments. This study builds on these efforts by proposing a lightweight unified computation module tailored for constrained devices.

B. PIPELINED AND MEMORY-EFFICIENT ARCHITECTURES

Memory management and parallel execution are critical for cryptographic acceleration. Multiple studies have explored pipelining, memory reuse, and BRAM-free architectures to mitigate resource limitations.

Bisheh-Niasar et al. [27] introduced a parallelized binomial sampler tightly integrated with a Keccak core, increasing the throughput of polynomial transformations for ML-KEM, reducing execution cycles, and improving overall performance efficiency.

Itabashi et al. [32] introduced a Proth-number–optimized modular polynomial multiplier that utilizes the K-RED algorithm for efficient modular reduction. The design incorporates the following:

- Radix-2, dual-parallel Radix-2, and Radix-4 hardware architectures to optimize execution time and area.
- A pipeline-optimized butterfly unit minimizes intermediate storage and latency.

Yaman et al. [33] developed three hardware configurations —high-performance (16 butterfly units), balanced (4 units), and lightweight (1 unit)—to investigate different computational trade-offs. They implemented a seven-stage butterfly process to ensure efficient memory access.

Recently, Javeed and Gregg [34] presented a complete NTT/INTT implementation for CRYSTALS-Kyber, emphasizing throughput and scalability; Wang et al. [35] introduced a scalable design featuring group-based pairwise memory access and rapid interstage reordering. Both architectures achieved high performance; however, they did not offer detailed area and power metrics for the address-generator submodule, making it difficult to compare with our approach.

These studies demonstrate the benefits of parallel processing and memory-efficient schemes. However, BRAM consumption continues to limit many FPGA-based solutions. This study improves scalability by employing a resource-efficient LUT-based data-access scheme throughout the design flow, eliminating the need for BRAM while maintaining computational performance.

C. MODULAR REDUCTION AND HARDWARE OPTIMIZATION TECHNIQUES

Modular reduction is pivotal in ML-KEM polynomial multiplication, leading numerous studies to develop hardware-efficient optimization techniques.

Bertels et al. [36] introduced a low-area modular reduction method utilizing 49 LUTs and a single DSP for modular multiplication. Bisheh-Niasar et al. [37] proposed a high-speed polynomial multiplication accelerator featuring a four-core butterfly system. This design enhances NTT and INTT throughput and employs K2-RED modular reduction to maintain 12-bit precision while reducing DSP dependency. Nguyen et al. [1] optimized the key generation, encapsulation, and decapsulation processes of ML-KEM via a single-buffer SHA3 hashing architecture. This design eliminated FIFO memory overhead and integrated dual-butterfly arithmetic to streamline execution.

Furthermore, Nguyen et al. [38] proposed tailored DSP-based modular multiplication units that eliminate the need for separate reduction steps by integrating modular reduction directly into the multiplication process. The proposed butterfly unit incorporates reduction operations, eliminating post-processing in the INTT. The unified point-wise modular multiplication architecture performs point-wise modular multiplications without necessitating extra memory.

Although these methods reduce modular arithmetic complexity, many prioritize performance gains at the expense of energy efficiency. By contrast, we propose a low-latency, area-optimized modular reduction unit specifically tailored for FPGA-based, energy-constrained implementations.

D. SUMMARY AND RESEARCH GAP

Despite significant advancements in FPGA-based ML-KEM, several limitations persist. In our proposed NTT architecture, we replaced BRAM primitives with distributed LUT-based RAM to enhance power efficiency, resource utilization, and memory-access performance. Unlike BRAM, LUT-RAM allows for localized placement within the FPGA fabric, prevents fragmentation in small storage scenarios, and reduces switching activity. These attributes collectively lower dynamic power consumption while maintaining throughput, making LUT-RAM an effective alternative to BRAM for small-to-medium memory structures in NTT operations.

Previous NTT architectures have primarily focused on multipliers and modular reduction, largely neglecting the address generator despite its crucial role in execution efficiency. To remedy this oversight, we introduce an optimized address generator that enhances memory access patterns and reduces logic overhead without utilizing BRAM or DSP resources.

Resource usage remains minimal by eliminating dependencies on BRAM and DSP. Furthermore, existing studies offer few FPGA implementations optimized for A^2T, with most concentrating on AT or AT^2 metrics. Our design specifically targets A^2T optimization to achieve a balance between energy efficiency and area minimization.

Additionally, by incorporating LUT-RAM-based memory and the optimized generator, this study addresses the following two gaps: (1) the lack of systematic address-generation strategies in NTT architectures and (2) the scarcity of A^2T-optimized FPGA designs. These advancements enhance the scalability and practical deployment of ML-KEM in constrained environments, such as IoT and embedded platforms.

IV. PROPOSED LIGHTWEIGHT NTT ARCHITECTURE

The proposed lightweight NTT module executes NTT/INTT operations with parameters $n = 256$ and $q = 3329$. Here, each polynomial is represented as a 256-element vector of 12-bit integers. Fig. 1 illustrates a distributed RAM (384 × 12 bits) featuring two read ports and one write port, which stores a 256-coefficient polynomial alongside 128 twiddle factors. The design exploits concurrent read operations to enhance throughput. A butterfly module performs modular addition, subtraction, and multiplication to facilitate the transformations.

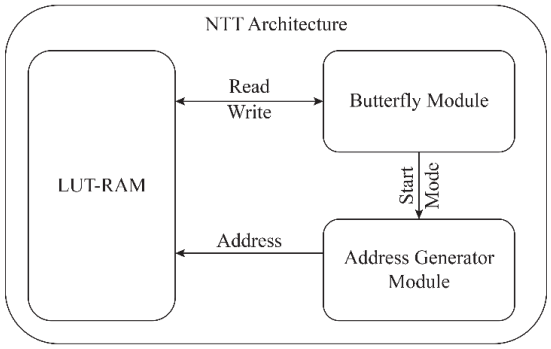

FIGURE 1. NTT architecture.

The module also incorporates an address generator that dynamically calculates memory addresses based on step and stage counts alongside the required operations. Given that butterfly operations involve nonlinear coefficient access, the address generator guarantees precise and timely data retrieval.

In the proposed architecture, distributed LUT-based RAM replaces block RAM (BRAM) primitives to enhance FPGA resource utilization, reduce power consumption, and eliminate dependence on vendor-specific BRAM configurations. Unlike BRAM, which can become fragmented and underutilized when accessed in small segments, LUT-RAM scales to meet the modest storage requirements of NTT operations. Distributing memory across the FPGA logic fabric allows for localized placement near computational units, reducing routing delays and improving timing closure. Additionally, LUT-RAM supports single-cycle access and can be configured for multiport operation, enabling parallel data movement during butterfly computations.

This strategy removes BRAM dependency, decreases dynamic power consumption, and maintains high-speed throughput, effectively balancing logic efficiency, power savings, and architectural scalability. The butterfly computations retrieve data from LUT-RAM under the control of the optimized address generator, ensuring efficient and synchronized memory access patterns. Distributed LUT-RAM also mitigates fragmentation and underutilization risks inherent to vendor-specific BRAM configurations.

Each butterfly computation executes in four clock cycles, with the resulting coefficient written back to the original LUT-RAM address to maintain storage consistency.

A. ADDRESS GENERATOR MODULE

This memory configuration achieves measurable dynamic power reduction compared to BRAM-based designs while sustaining the requisite throughput for NTT implementations. The address generator is a critical component of the proposed NTT module, handling data read and write operations at specified memory locations. It ensures optimized memory access, efficient address mapping, and reduced implementation costs. Traditional NTT address generators often rely on complex computations or extensive look-up tables, which increase area, memory, and energy consumption.

To overcome these challenges, we introduce a lightweight, resource-efficient address generator. The design utilizes a 10-bit incrementing counter enhanced with 3-bit XOR logic on the most significant bits to enable partial bit reversal, alongside multiplexers. Single-bit start and mode inputs distinguish between forward and inverse NTT operations. The generator simultaneously produces three addresses (ports A, B, and C) for polynomial coefficients and twiddle factors corresponding to each computation step.

All NTT and INTT transformations consist of seven stages with 128 steps each; thus, a 10-bit counter is sufficient to encode step indices within each stage. According to Algorithm 1, each step generates addresses for coefficients (A, B) and twiddle factors (TW), which are dynamically computed by the 10-bit counter and adjusted based on the current step count, stage count, and NTT mode.

Although previous research has predominantly focused on multipliers and modular reduction—the main computational bottlenecks in NTT—the address generator has remained relatively underexplored. Efficient address generation is crucial for optimizing memory access patterns, pipeline throughput, and overall hardware utilization. In this study, we introduce a lightweight, power-efficient address generator specifically designed for both forward and inverse NTT.

Algorithm 1 Address Generator Module

Inputs: *Start:* 1-bit control signal; *Operation:* 1-bit mode signal (0 for forward NTT, 1 for inverse NTT);
Outputs: *A, B, TW*: 9-bit memory addresses;
Begin:
Initialize *Counter* [10] as a 10-bit zero counter
Set j= *Counter*[6: 0] ▷ Extract lower 7 bits of the counter
if *Operation*= 0 **then**
  Set i = *Counter* [9:7]
  ▷ Extract upper 3 bits for forward NTT
else
  Set i= {(*Counter*[9:7] $\oplus$ 7) − 1 }
  ▷Modify upper 3 bits for inverse NTT
end if
while *Start*/= 0 **do**
  if i = 0 **then**
    for j= 0 **to**127 **do**
      $A \leftarrow$ *concatenate* {0, 0, j [6:0]}
      ▷ Generate address for coefficient A
      $B \leftarrow$ *concatenate* {0, 1, j [6:0]}
      ▷ Generate address for coefficient B
      $TW \leftarrow$ *concatenate* {1, 0, 1, 0, 0, 0, 0, 0, 0}
      ▷ Twiddle factor address
    end for
  else
    for i = 1 **to** 6 **do**
    for j= 0 **to** 127 **do**
      $A \leftarrow$ *concatenate* {0, j[6: (7 − i)], 0, j [(6 − i): 0]} ▷ Coefficient Aaddress
      $B \leftarrow$ *concatenate* {0, j[6: (7 − i)], 1, j [(6 − i): 0]} ▷ Coefficient Baddress
      $TW \leftarrow$ *concatenate* {1, 0, j[(7 − i): 6], (1 $\ll$ (6 − i))} ▷ Twiddle factor address
      end for
    end for
    end if
  Increment *Counter* ▷ Move to the next step
end while
End

These addresses are derived from bit-concatenation patterns that adapt based on the stage index and NTT mode. This method eliminates the need for large LUTs and complex computational units used in previous designs, thereby reducing LUT usage, BRAM requirements, and overall power consumption.

1) DESIGN OVERVIEW

The proposed architecture utilizes a 10-bit counter to manage seven NTT stages, each comprising 128 butterfly operations. The lower seven bits (Counter[6:0]) represent intra-stage indices, whereas the upper three bits (Counter[9:7]) denote the active stage. A streamlined XOR block modifies the most significant bit (MSB) segment in inverse mode, producing correctly reversed address sequences without requiring additional hardware.

2) MULTIPORT ADDRESS OUTPUT

At each computation step, the module generates three addresses:

- Port A for coefficient A,
- Port B for coefficient B,
- Port TW for twiddle factors.

3) OPERATIONAL FLOW

In Stage 1, the addresses for Port A and Port B differ by one bit to facilitate paired butterfly operations, whereas the Port TW addresses remain constant throughout each stage iteration. Between Stages 2 and 7, the block diagram shows shifts in address-increment bit groupings, enabling memory reuse without collisions and supporting all 128 operations per stage, thereby minimizing storage requirements.

4) KEY ADVANTAGES

Hardware Efficiency: Achieves address generation with minimal logic (counters + XOR gates + multiplexers), which reduces both area and power consumption.

Dual-Mode Functionality: Single design supports both forward and inverse NTT without duplication.

Integrating the address generator into the NTT pipeline results in low-area usage, low power consumption, and high throughput in FPGA-based lattice cryptography accelerators, which addresses a critical but often neglected aspect of design.

B. BUTTERFLY MODULE

The butterfly unit executes modular arithmetic for the NTT and its inverse (INTT) using two ML-KEM algorithms: Cooley–Tukey (CT) for forward NTT and Gentleman–Sande (GS) for inverse NTT. These algorithms perform modular addition, subtraction, and multiplication with precomputed twiddle factors; however, they differ in execution order and data arrangement.

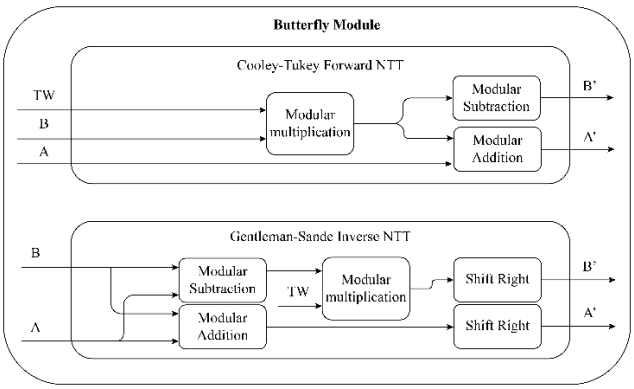

FIGURE 2. Butterfly module.

Previous NTT hardware designs primarily optimized modular multipliers and reduction algorithms to balance area and speed, often at the expense of power and area efficiency. By contrast, this study emphasizes butterfly unit optimization, as its efficiency directly impacts performance and FPGA resource utilization.

This streamlined butterfly unit amalgamates Cooley–Tukey (CT) and Gentleman–Sande (GS) algorithms into a single hardware block, utilizing shared resources for modular addition, subtraction, and multiplication. This integration eliminates the need for duplicating arithmetic units, reducing the FPGA area. Additionally, since CT produces bit-reversed outputs and GS requires bit-reversed inputs, the separate bit-reversal stage is entirely omitted. Drawing inspiration from Zhang et al.'s study on *efficient reconfigurable modular multipliers for post-quantum digital signatures*, the post-INTT scaling step—which conventionally demands 256 modular multiplications—is eliminated. The accuracy is maintained by conditionally adding q to odd inputs prior to the right shift operation, which decreases the number of necessary multiplications by 22, thus lowering both latency and hardware costs.

The butterfly module features a modular multiplication unit, recognized as the most hardware-intensive component owing to the complexity of modular reduction. To mitigate this, we implement Bertels et al.'s compact and efficient design [36], which provides low area and latency through optimizations tailored for resource-constrained devices. Bertels et al.'s approach builds upon the advancements of Nguyen et al. [38] and Ni et al. [39], utilizing LUT-6 primitives and DSPs to optimize the balance between implementation cost and computational efficiency. Enhancements in efficiency include reducing conditional additions, employing LUT-based reduction during low-precision phases, and applying K-reduction to limit input ranges. These strategies achieve the implementation using only 49 LUTs while sustaining robust performance.

As illustrated in Fig. 2, the butterfly module seamlessly integrates the Cooley–Tukey forward NTT and Gentleman–Sande inverse NTT within a unified architecture. Modular addition, subtraction, and multiplication units are employed for both GS and CT algorithms, with execution sequences aligned to the transform direction. This shared resource approach decreases logic usage, switching activity, and power consumption. The GS pathway integrates a right-shift stage with conditional addition of q, maintaining modular correctness without requiring a separate scaling phase. By removing the bit-reversal stage and post-INTT scaling multiplications and adopting a streamlined modular reduction method, latency is reduced, area is minimized, and energy efficiency is enhanced—providing an optimized throughput–resource balance for FPGA-based ML-KEM.

To further minimize hardware costs, the same arithmetic units are shared for modular addition, subtraction, and multiplication in both GS and CT operations, where only the execution order differs. In the CT operation (4), modular multiplication occurs before addition and subtraction; conversely, GS performs addition and subtraction before multiplication (5).

$$aout = (ain + bin \cdot w)\ mod\ q,$$
$$bout = (ain - bin \cdot w)\ mod\ q \tag{4}$$
$$aout = (ain + bin) \cdot w\ mod\ q,$$
$$bout = (ain - bin) \cdot w\ mod\ q \tag{5}$$

The proposed lightweight multicycle architecture consolidates forward and inverse NTT into a single shared butterfly engine. The same modular addition, subtraction, and multiplication units are reused across both transforms, minimizing FPGA resource utilization. The differentiation between forward and inverse operations is achieved purely through operation scheduling over four-cycle executions, avoiding hardware duplication. Each butterfly unit operates across four cycles using dual-port asynchronous LUT-RAM, thus minimizing FPGA resource utilization. Each cycle directs ports to designated operand addresses, enabling asynchronous reads or preparing specific addresses for subsequent write-back operations. This fine-grained scheduling ensures efficient memory access while maintaining low-area and power consumption.

In the forward NTT, illustrated in Fig. 3, operands from addresses A and B are fetched and readied for computation. During the second cycle, port A is reassigned to the twiddle factor address, whereas port B continues supplying operand B, which is then multiplied by the twiddle factor. In the third cycle, operand A is prefetched, and modular addition or subtraction operations are performed with the multiplication result, generating the first butterfly output and writing it back to address A. In the final cycle, the second butterfly output is written to address B. This streamlined timing schedule completes the butterfly computation in four cycles without duplicating arithmetic units.

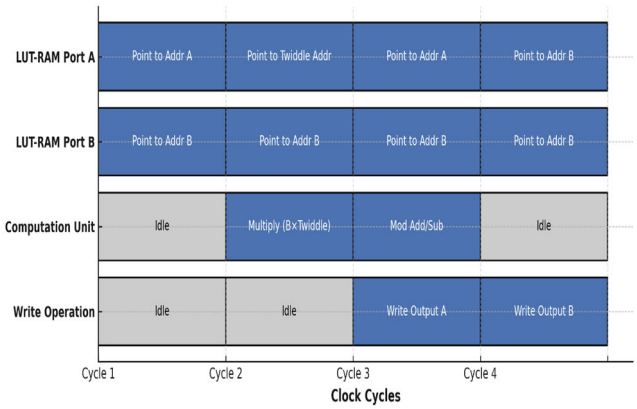

FIGURE 3. NTT timing diagram.

The inverse NTT, shown in Fig. 4, employs a similar yet distinct cycle assignment. In the first cycle, operands A and B are retrieved; a modular subtraction operation is executed, and the result is buffered for the next stage. In the second cycle, port A is reassigned to the twiddle factor address, and the subtraction result is multiplied by the corresponding twiddle factor. In the third cycle, operand A is fetched again; a modular addition operation is performed and written back to address A. In the final cycle, the multiplication result from the second cycle is written to address B. The scheduling ensures inverse transformation within the same four-cycle hardware budget as the forward process by integrating CT and GS butterflies into a unified execution flow. This integration eliminates arithmetic-unit duplication, reduces hardware overhead, and ensures consistent timing.

The LUT-RAM-based multicycle scheduling enhances energy efficiency and enables scalable performance for both forward and inverse transformations in post-quantum cryptographic applications (Fig. 5). This does not introduce a new butterfly algorithm; instead, it entails redesigning and re-architecting the current state-of-the-art butterfly module to improve area and power efficiency while maintaining throughput. The design employs established modular arithmetic techniques and optimized hardware mapping strategies.

The key hardware-level refinements include a multicycle architecture that avoids pipelining, decreases flip-flop usage, and lowers dynamic power by restricting the number of simultaneously active resources. Computation

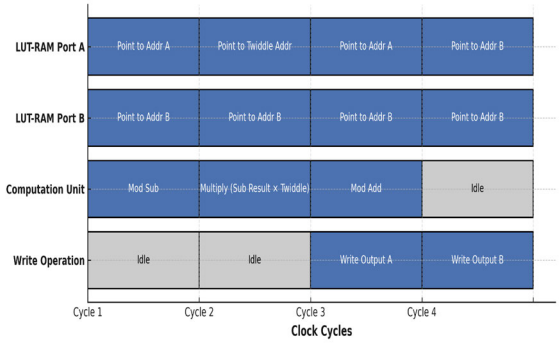

FIGURE 4. INTT timing diagram.

aligns shared arithmetic units with optimized memory-access patterns to achieve competitive throughput and minimal FPGA utilization. A register-layer removal strategy within the modular multiplication unit eliminates registers in the post-multiplication and K-RED stages, thereby reducing hardware overhead and enhancing efficiency.

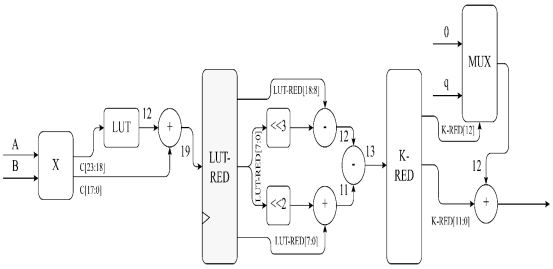

FIGURE 5. Modular reduction unit [36].

This modification allows single-cycle computation instead of three, decreasing flip-flop usage and cycle latency. Although longer combinational paths may marginally reduce the maximum frequency, this trade-off lowers power consumption and simultaneously improves effective throughput. Additionally, DSP blocks are replaced with LUT-based multiplication. The multiplier is entirely implemented using LUTs, avoiding DSP primitives that constrain portability and consume fixed device resources. Vivado's synthesis engine optimizes placement and routing within the constraints of this architecture, leading to enhanced resource distribution, reduced power consumption, and increased portability across various FPGA families.

V. RESULTS AND DISCUSSION

This section presents an evaluation of the performance of the proposed lightweight NTT implementation by analyzing FPGA resource utilization (slices, LUTs, DSPs, and BRAMs), operating frequency, cycle count, execution time, throughput efficiency, and A^2T values; these results are compared with those of prior works summarized in Table 1. Table 2 presents both static and dynamic power measurements to assess the energy efficiency of designs that either report power data or provide source code enabling measurements on the Virtex-7 FPGA. The reported

TABLE 1. Comparison of implementation results for the ntt architecture.

Paper	Total Area Cost[1]	Slices[2]	LUT	FF	DSP	BRAM	frequency in (MHz)	Cycles	Time (us)	Throughput[3] (Kops/sec/slice)	A^2T	Total Area Cost Enhancement
This Work	147	147	557	59	0	0	100	3584	35.84	0.1898	7.75E+05	0%
[1]	827	327	473	599	2	1.5	227	448	1.97	1.5523	1.35E+06	82.22%
[19]	1021	221	541	680	0	0	417	461	1.11	4.0765	1.15E+06	85.60%
[27]	809	109	360	145	3	2	115	940	8.17	1.1229	5.35E+06	81.83%
[28]	1579	1579	4731	3169	0	0	187	1024	5.48	0.1156	1.37E+07	90.69%
[32]	492	92	274	181	1	1.5	208	902	4.34	2.5045	1.05E+06	70.12%
[33]	881	281	948	352	1	2.5	190	904	4.76	0.7476	3.69E+06	83.31%
[36]	825	225	690	419	0	2.5	190	904	4.76	0.9337	3.24E+06	82.18%
[37]	1090	1090	801	717	0	2	222	324	1.46	0.6284	1.73E+06	86.51%
[39]	883	283	696	869	1	2.5	215	1024	4.76	0.747	3.12E+06	83.35%
[34]	812	212	645	478	0	2.0	250	1024	4.10	1.152	4.82E+06	81.89%
[35]	905	305	910	510	1	2.0	280	1024	3.66	1.258	5.20E+06	83.75%
[38]	784	284	824	625	0	2.5	283	1024	3.62	1.272	4.95E+06	81.25%
[31]	831	231	700	447	1	2.5	161	1024	6.36	0.676	2.96E+06	82.31%

[1] The total area cost is estimated using (slice + DSP × 100 + BRAM × 200) [40].
[2] The slice was estimated using the formula (number of LUTs/4 + number of FF/8), as one slice consisted of four LUTs and eight FF in Artix-7 [34].
[3] Throughput per Slice = Throughput (Kops/sec) / Number of total Slices

metrics—FPGA resource utilization, operating frequency, execution time, and A^2T—underscore the suitability of the proposed architecture for deployment in resource-constrained devices.

The proposed implementation occupies only 147 slices and 557 LUTs, completely eliminating the need for DSPs and BRAMs and significantly reducing the overall area cost. Compared to [19], which reports a total area cost of 1021 using 221 slices, 541 LUTs, 680 FFs, and 4 BRAMs, the proposed design achieves an 85.6% area reduction and a 1.49× improvement in A^2T efficiency (7.745×10^5 *vs.* 1.152×10^6). Similarly, compared to [27], which reports a total area cost of 809 using 109 slices, 360 LUTs, 145 FFs, 3 DSPs, and 2 BRAMs, this design realizes an 81.8% area reduction and a 6.91× improvement in A^2T value (7.745×10^5 vs. 5.350×10^6).

Table 1 provides a comprehensive comparison of thirteen previous NTT architectures alongside the proposed design, detailing FPGA resource utilization, operating frequency, cycle count, execution time, and throughput efficiency. The proposed design exhibits superior area and resource efficiency, achieving the lowest total area cost (147) as a zero-overhead baseline. Alternative implementations, such as [19] (1021) [28], (1579), and [37] (1090), require 7×–10× larger area. Even the most competitive counterpart in [32] consumes over three times the area of the proposed design.

Regarding logic resources, the proposed design utilizes 557 LUTs and 59 FFs without DSPs or BRAMs, whereas [31], [33], [35], [36] employ 700–950 LUTs, 350–550 FFs, and depend on DSPs and BRAMs. By eliminating DSPs and BRAMs, the proposed design enhances portability to low-cost FPGAs and removes vendor-specific dependencies. Additionally, the implementation in [28] demands an order of magnitude more LUTs (4,731) and FFs (3,169), underscoring the lightweight nature of the proposed architecture.

The operating frequency of 100 MHz remains competitive, though lower than the 115–227 MHz achieved by [1], [27], [32], [33], and [36] at the expense of a larger area. When normalized, the proposed design attains 0.1898 Kops/sec/slice. Although this value is lower than the throughput-focused designs in [27] (1.1229) [19], (4.0765) and [34] (1.152), it reflects a balanced trade-off favoring compactness and energy efficiency.

TABLE 2. Detailed design utilization and power consumption on (virtex-7).

Paper	Module	LUT	FF	DSP	BRAM	Dynamic Power Consumption (mW)	Total Power Consumption (Static and Dynamic) (mW)
This Work	**NTT Top Module**	**557**	**59**	**0**	**0**	**15**	**255**
	|Memory Modules	275	0	0	0	5	
	|Address Generator Modules	49	28	0	0	8	
	|Butterfly Modules	233	31	0	0	2	
Xing and Li [31]	**NTT Top Module**	**700**	**447**	**2.5**	**1**	**39**	**281**
	|Memory Modules	44	0	2.5	0	11	
	|Address Generator Modules	445	294	0	0	17	
	|Butterfly Modules	211	153	0	1	11	
Nguyen et al. [38]	**NTT Top Module**	**824**	**625**	**2.5**	**0**	**44**	**287**
	|Memory Modules	40	0	2.5	0	11	
	|Address Generator Modules	452	298	0	0	21	
	|Butterfly Modules	332	327	0	0	12	
Bertels et al. [36]	**NTT Top Module**	**661**	**846**	**2.5**	**1**	**40**	**282**
	|Memory Modules	26	0	2.5	0	11	
	|Address Generator Modules	421	613	0	0	15	
	|Butterfly Modules	214	233	0	1	14	
Ni et al. [39]	**NTT Top Module**	**696**	**869**	**2.5**	**1**	**53**	**295**
	|Memory Modules	26	0	2.5	0	11	
	|Address Generator Modules	421	613	0	0	19	
	|Butterfly Modules	249	256	0	1	23	
Yaman et al. [33]	**NTT Top Module**	**746**	**542**	**2.5**	**1**	**57**	**299**
	|Memory Modules	43	0	2.5	0	12	
	|Address Generator Modules	445	298	0	0	32	
	|Butterfly Modules	258	244	0	1	13	

The execution time required 35.84 μs to complete an NTT operation. In the experimental evaluation, normalized by area, our performance was competitive with [19] (1.11 μs) [27], (8.17 μs) and [38] (3.62 μs), whereas FPGA overhead was significantly lower. The total area cost enhancement metric validated our architectural efficiency, showing 0% overhead compared to prior designs that incurred at least 70% higher costs, with [28], [37] exceeding 85%. These results demonstrate that the proposed architecture achieves best-in-class resource efficiency, making it ideal for post-quantum cryptographic implementations on resource-constrained FPGAs. Although some architectures offered higher raw throughput, they demanded disproportionately higher area and power. Our lightweight, multi-cycle, and DSP–BRAM-free design provides a balanced solution that minimizes the hardware footprint while maintaining competitive performance.

Table 2 presents a detailed comparative analysis of dynamic power consumption across five implementations. The proposed design consumed only 15 mW of dynamic power, which is significantly lower than those of [33] (57 mW) [39], (53 mW), and [36] (40 mW). This reduction is primarily because of three key architectural factors: (1) a lightweight address generator module; (2) a multi-cycle, BRAM-free, optimized unified-butterfly module; and (3) LUT-based modular multiplication that eliminates dependence on DSP resources.; and (3) LUT-based modular multiplication that removes dependence on DSP resources.

Specifically, module-level analysis revealed that the butterfly unit consumed merely 2 mW, compared to 13 mW in [33] and 12 mW in [38]. Similarly, the address generator used 8 mW versus 19 mW in [39], representing an order-of-magnitude improvement. Even memory modules consumed only 5 mW, compared to approximately 11–12 mW consistently reported in [31], [33], [36], [38], and [39]. These reductions demonstrate that the architecture maintains high performance while significantly lowering dynamic power consumption.

A^2T (Area Squared $\times$ Time) is a key efficiency metric that balances hardware resource utilization and latency. Lower A^2T values signify enhanced aggregate efficiency, especially for FPGA-based implementations in resource-constrained

environments. In addition to achieving a best-in-class A^2T of 7.75×10^5, the proposed design shows that a smaller hardware footprint directly reduces power usage. The total area savings from eliminating DSPs and BRAMs and decreasing LUT and slice counts improve A^2T and minimize switching activity, corresponding to the 15 mW dynamic and 275 mW total (static + dynamic) power shown in Table 2. Prior works [33], [36], [39] consume 40–57 mW of dynamic power. By contrast, the proposed architecture achieves both efficiency and lower energy demand. This consistency between area efficiency and power results validates the assumption that compact designs inherently consume less energy. Unlike high-frequency designs such as [19] (417 MHz), our approach prioritizes energy efficiency by operating at 100 MHz, making it especially suitable for IoT and edge-computing applications where reducing energy consumption is more critical than maximizing throughput.

The complete elimination of DSPs and BRAMs enhances scalability across diverse FPGA platforms by reducing dependence on vendor-specific resources; this enables portability to lower-cost FPGAs and facilitates broader adoption in constrained environments. The proposed architecture stands apart from existing designs that rely on specialized resources by significantly reducing both static and dynamic energy consumption, making it ideal for environments with hardware and energy limitations; this allows deployment on compact, resource-constrained devices without compromising efficiency.

By emphasizing A^2T, we prioritize minimal area usage while maintaining acceptable execution times. Although higher-performance implementations such as [19] achieve faster execution speeds, our architecture focuses on area optimization to reduce energy usage. This approach is aligned with applications where energy efficiency and hardware compactness are more critical than maximum throughput.

Energy savings are achieved by eliminating high-consumption components and optimizing resource utilization, as demonstrated by both static and dynamic power measurements. Our implementation showcases an energy-efficient design tailored for constrained environments, ensuring its broad applicability in post-quantum cryptographic scenarios.

Theoretically, these findings advance the development of energy-efficient post-quantum architectures, particularly for lattice-based schemes such as ML-KEM. By optimizing the overclocked address-generator module and enhancing the butterfly module with a multi-cycle design that eliminates BRAM usage and removes DSPs—thereby improving power efficiency and ensuring portability across FPGA vendors—this study demonstrates a pathway to secure computation without compromising scalability and provides valuable guidance for future FPGA-based accelerators that prioritize minimal resource consumption.

Practically, our design presents a feasible solution for edge devices, IoT nodes, and embedded systems where energy efficiency, minimal area, and hardware simplicity are paramount. These features facilitate secure communication within constrained environments, particularly in smart healthcare, industrial automation, and environmental monitoring, where lightweight and efficient security solutions are crucial. These sectors require security measures that consume minimal resources due to limited computational and power capacities.

VI. CONCLUSION AND FUTURE WORK

This study introduces a lightweight and efficient NTT implementation for ML-KEM to address the demand for secure cryptographic algorithms in resource-constrained devices. The proposed design eliminates the need for DSP and BRAM in the unified butterfly module and utilizes an efficient address generator, resulting in a low area cost of 147 slices and a highly efficient A^2T value of 7.745×105. These optimizations make the architecture highly suitable for IoT and edge-computing environments. Compared to existing works, the proposed design exhibits substantial resource-efficiency enhancements, attaining up to an 88.8% reduction in total area cost and a $17.57\times$ improvement in A^2T.

Although this study primarily focuses on area and power efficiency, the elimination of DSPs and BRAMs also results in significant energy savings. This alignment between area minimization and expected power reduction highlights the suitability of the proposed architecture for constrained environments, ensuring compatibility across various FPGA platforms.

Future work will include direct energy evaluation via FPGA-based physical implementations. This strategy will clarify power–area trade-offs. Such an analysis will provide a comprehensive assessment of the relationship between area efficiency and power savings under realistic operating conditions and help new researchers implement countermeasures against side-channel attacks. Another direction entails investigating the design using a standard-cell ASIC flow, as the architecture is independent of FPGA-specific primitives. This approach would permit more accurate power, area, and timing characterizations and extend its applicability beyond FPGA platforms.

Integrating FPGA-based prototyping with ASIC-oriented analysis will provide deeper insights into optimizing lightweight NTT architectures for both reconfigurable and dedicated hardware targets in post-quantum cryptography. Furthermore, adapting the architecture to other lattice-based PQC schemes beyond ML-KEM will ensure broader applicability to evolving PQC standards. By reusing the lightweight address generator and unified butterfly architecture, and tailoring parameters to specific algorithms, we anticipate maintaining low-area and low-power benefits across a wider range of cryptographic applications.

ACKNOWLEDGMENT

The authors would like to express their gratitude to WAQF and the Deanship of Scientific Research (DSR) for their technical.

REFERENCES

[1] T.-H. Nguyen, D.-T. Dam, P.-P. Duong, B. Kieu-Do-Nguyen, C.-K. Pham, and T.-T. Hoang, "Efficient hardware implementation of the lightweight CRYSTALS-kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 2, pp. 610–622, Feb. 2025, doi: 10.1109/TCSI.2024.3443238.

[2] H. Delfs and H. Knebl, *Introduction To Cryptography: Principles and Applications*, 2nd ed., Berlin, Germany: Springer, 2007, doi: 10.1007/3-540-49244-5.

[3] L. Liu and M. T. Özsu, *Encyclopedia of Database Systems*, vol. 6. Cham, Switzerland: Springer, 2009, doi: 10.1007/978-0-387-39940-9.

[4] C.-l. Chan, R. Fontugne, K. Cho, and S. Goto, "Monitoring TLS adoption using backbone and edge traffic," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Apr. 2018, pp. 208–213, doi: 10.1109/INFCOMW.2018.8406957.

[5] D. Naylor, A. Finamore, I. Leontiadis, Y. Grunenberger, M. Mellia, M. Munafò, K. Papagiannaki, and P. Steenkiste, "The cost of the 'S' in HTTPS," in *Proc. 10th ACM Int. Conf. Emerg. Netw. Experiments Technol.*, Dec. 2014, pp. 133–140, doi: 10.1145/2674005.2674991.

[6] G. Fitzgibbon and C. Ottaviani, "Constrained device performance benchmarking with the implementation of post-quantum cryptography," *Cryptography*, vol. 8, no. 2, p. 21, 2024, doi: 10.3390/cryptography8020021.

[7] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, Santa Fe, NM, USA, 1994, pp. 124–134, doi: 10.1109/sfcs.1994.365700.

[8] D. J. Bernstein, "Introduction to post-quantum cryptography," in *Proc. Post-Quantum Cryptography*, Jan. 2009, pp. 1–14, doi: 10.1007/978-3-540-88702-7_1.

[9] W. Buchanan and A. Woodward, "Will quantum computers be the end of public key encryption?" *J. Cyber Secur. Technol.*, vol. 1, no. 1, pp. 1–22, Jan. 2017, doi: 10.1080/23742917.2016.1226650.

[10] *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, Standard FIPS 140-3, National Institute of Standards and Technology, 2024, doi: 10.6028/NIST.FIPS.203.

[11] H. Houssain, "Elliptic curve cryptography algorithms resistant against power analysis attacks on resource-constrained devices," Ph.D. dissertation, Doctoral School Eng. Sci. Clermont-Ferrand Univ., Dept. Comput. Sci., Blaise Pascal Clermont Ferrand II University, Aubière, France, 2012.

[12] T. F. Al-Somani, "Very efficient point multiplication on Koblitz curves," *IEICE Electron. Exp.*, vol. 13, no. 9, 2016, Art. no. 20160044, doi: 10.1587/elex.13.20160044.

[13] C. D. Thompson, "A complexity theory for VLSI," Ph.D. dissertation, Dept. Comput. Sci., Carnegie Mellon University, Pittsburgh, PA, USA, 1980.

[14] M. Rashid, L. Ardito, and M. Torchiano, "Energy consumption analysis of algorithms implementations," in *Proc. ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*, Oct. 2015, pp. 1–4, doi: 10.1109/ESEM.2015.7321198.

[15] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, "Energy efficiency across programming languages: How do energy, time, and memory relate?" in *Proc. 10th ACM SIGPLAN Int. Conf. Softw. Lang. Eng.*, Oct. 2017, pp. 256–267, doi: 10.1145/3136014.3136031.

[16] A. Langlois and D. Stehlé, "Worst-case to average-case reductions for module lattices," *Designs, Codes Cryptography*, vol. 75, no. 3, pp. 565–599, Jun. 2015, doi: 10.1007/s10623-014-9938-4.

[17] E. Fujisaki and T. Okamoto, "Secure integration of asymmetric and symmetric encryption schemes," *J. Cryptol.*, vol. 26, no. 1, pp. 80–101, Jan. 2013, doi: 10.1007/s00145-011-9114-1.

[18] D. Hofheinz, K. Hövelmanns, and E. Kiltz, "A modular analysis of the Fujisaki-Okamoto transformation," in *Theory of Cryptography*. Cham, Switzerland: Springer, 2017, pp. 341–371.

[19] B. Kieu-Do-Nguyen, N. The Binh, C. Pham-Quoc, H. P. Nghi, N.-T. Tran, T.-T. Hoang, and C.-K. Pham, "Compact and low-latency FPGA-based number theoretic transform architecture for CRYSTALS kyber postquantum cryptography scheme," *Information*, vol. 15, no. 7, p. 400, Jul. 2024, doi: 10.3390/info15070400.

[20] M. Das and B. Jajodia, "19th international symposium on design and optimization (ISOCC)," in *Proc. 19th Int. Symp. Design Optim. (ISOCC)*, Jan. 2022, pp. 65–66, doi: 10.1109/ISOCC56007.2022.10031366.

[21] Y. A. Birgani, S. Timarchi, and A. Khalid, "Area-time-efficient scalable schoolbook polynomial multiplier for lattice-based cryptography," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 12, pp. 5079–5083, Dec. 2022, doi: 10.1109/TCSII.2022.3188943.

[22] S. Yang, D. Liu, A. Hu, A. Li, J. Zhang, X. Li, J. Lu, and C. Mo, "An instruction-configurable post-quantum cryptographic processor towards NTRU," in *Proc. Asian Hardw. Oriented Secur. Trust Symp. (AsianHOST)*, Dec. 2022, pp. 1–6, doi: 10.1109/AsianHOST56390.2022.10022178.

[23] Z.-Y. Wong, D. C.-K. Wong, W.-K. Lee, K.-M. Mok, W.-S. Yap, and A. Khalid, "KaratSaber: New speed records for saber polynomial multiplication using efficient Karatsuba FPGA architecture," *IEEE Trans. Comput.*, pp. 1–13, Jul. 2023, doi: 10.1109/TC.2023.3238129.

[24] A. Ghosh, J. M. B. Mera, A. Karmakar, D. Das, S. Ghosh, I. Verbauwhede, and S. Sen, "A 334 μW 0.158 mm^2 ASIC for post-quantum key-encapsulation mechanism saber with low-latency striding Toom–Cook multiplication," *IEEE J. Solid-State Circuits*, vol. 58, no. 8, pp. 2383–2398, Aug. 2023, doi: 10.1109/JSSC.2023.3253425.

[25] J. Wang, C. Yang, F. Zhang, Y. Meng, and Y. Su, "TCPM: A reconfigurable and efficient Toom–Cook-based polynomial multiplier over rings using a novel compressed postprocessing algorithm," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 8, pp. 1153–1166, Aug. 2023, doi: 10.1109/TVLSI.2023.3277865.

[26] J. M. Pollard, "The fast Fourier transform in a finite field," *Math. Comput.*, vol. 25, no. 114, pp. 365–374, 1971, doi: 10.1090/s0025-5718-1971-0301966-0.

[27] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "Instruction-set accelerated implementation of CRYSTALS-kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 11, pp. 4648–4659, Nov. 2021, doi: 10.1109/TCSI.2021.3106639.

[28] A. Yahya Hummdi, A. Aljaedi, Z. Bassfar, S. Shaukat Jamal, M. Mazyad Hazzazi, and M. U. Rehman, "Unif-NTT: A unified hardware design of forward and inverse NTT for PQC algorithms," *IEEE Access*, vol. 12, pp. 94793–94804, 2024, doi: 10.1109/ACCESS.2024.3425813.

[29] K. Javeed, S. Rubab, and D. Gregg, "Efficient reconfigurable modular multipliers for post-quantum digital signatures," in *Proc. 31st IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, Nov. 2024, pp. 1–4, doi: 10.1109/ICECS61496.2024.10849189.

[30] Y. A. Shah, C. Rafferty, A. Khalid, S. Khan, K. Javeed, and M. O'Neill, "Efficient soft core multiplier for post quantum digital signatures," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2024, pp. 1–5, doi: 10.1109/ISCAS58744.2024.10558234.

[31] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA," *TCHES*, vol. 2021, no. 2, pp. 328–356, Feb. 2021, doi: 10.46586/tches.v2021.i2.328-356.

[32] Y. Itabashi, R. Ueno, and N. Homma, "Efficient modular polynomial multiplier for NTT accelerator of crystals-kyber," in *Proc. 25th Euromicro Conf. Digit. Syst. Design (DSD)*, Maspalomas, Spain, Aug. 2022, pp. 528–533, doi: 10.1109/DSD57027.2022.00076.

[33] F. Yaman, A. C. Mert, E. Öztürk, and E. Savas, "A hardware accelerator for polynomial multiplication operation of CRYSTALS-KYBER PQC scheme," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1020–1025, doi: 10.23919/DATE51398.2021.9474139.

[34] K. Javeed and D. Gregg, "Efficient number theoretic transform architecture for CRYSTALS-kyber," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 72, no. 1, pp. 263–267, Jan. 2025, doi: 10.1109/TCSII.2024.3465273.

[35] Z. Wang, Y. Yang, J. Wang, J. Hou, Y. Su, and C. Yang, "A scalable and efficient NTT/INTT architecture using group-based pairwise memory access and fast interstage reordering," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 2, pp. 588–592, Feb. 2025, doi: 10.1109/TVLSI.2024.3465010.

[36] J. Bertels, Q. Norga, and I. Verbauwhede, "A better kyber butterfly for FPGAs," in *Proc. 34th Int. Conf. Field-Program. Log. Appl. (FPL)*, Sep. 2024, pp. 171–177, doi: 10.1109/fpl64840.2024.00032.

[37] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "High-speed NTT-based polynomial multiplication accelerator for post-quantum cryptography," in *Proc. IEEE 28th Symp. Comput. Arithmetic (ARITH)*, Lyngby, Denmark, Jun. 2021, pp. 94–101, doi: 10.1109/ARITH51176.2021.00028.

[38] T.-H. Nguyen, C.-K. Pham, and T.-T. Hoang, "A high-efficiency modular multiplication digital signal processing for lattice-based post-quantum cryptography," *Cryptography*, vol. 7, no. 4, p. 46, Sep. 2023, doi: 10.3390/cryptography7040046.

[39] Z. Ni, A. Khalid, W. Liu, and M. O'Neill, "Towards a lightweight CRYSTALS-kyber in FPGAs: An ultra-lightweight BRAM-free NTT core," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Monterey, CA, USA, May 2023, pp. 1–5, doi: 10.1109/ISCAS46773.2023.10181340.

[40] *7 Series FPGAs Data Sheet: Overview*, Xilinx Corp., San Jose, CA, USA, 2024.

MOHAMMED O. ALSHOMRANY received the B.Sc. degree in computer engineering from Umm Al-Qura University, Mecca, Saudi Arabia, in 2013, and the M.Sc. degree in information security from Saudi Electronic University, Riyadh, Saudi Arabia, in 2020. He is currently pursuing the Ph.D. degree in computer science with King Abdulaziz University, Saudi Arabia.

His research interests include hardware security, microarchitectural defenses against side-channel attacks, and the hardware acceleration of lattice-based post-quantum cryptography, with an emphasis on secure processor design and the implementation of quantum-resistant algorithms.

MUHAMMAD AL-HASHIMI received the B.S. degree in electrical engineering (electronics and communications) from King Abdulaziz University, Jeddah, Saudi Arabia, in 1987, and the M.S. and Ph.D. degrees in computer science from Texas A&M University, College Station, TX, USA, in 1993 and 2000, respectively.

He is currently an Assistant Professor with the Department of Computer Science, King Abdulaziz University, and a member of the High-Performance Computing (HPC) Research Group. As the Vice Dean for Development, he was responsible for implementing quality management systems for undergraduate programs. His research is focused on energy-efficient computing within high-performance systems.

FAWAZ ALSOLAMI holds the position of an Associate Professor of computer science with King Abdulaziz University, Saudi Arabia. He has extensively published in prestigious IEEE journals, including IEEE Transactions on Multimedia, IEEE Transactions on Intelligent Transportation Systems, and IEEE Internet of Things Journal. Additionally, he has led more than ten funded research projects. His research interests include artificial intelligence, machine learning, and combinatorial optimization.

MOSTAFA SALEH received the B.S., M.S., and Ph.D. degrees in computer engineering from Mansoura University, Mansoura, Egypt, in 1992, 1995, and 2000, respectively.

He is currently a Professor with the Faculty of Computing and Information Technology, King Abdulaziz University, Jeddah, Saudi Arabia, and is a member of the High-Performance Computing (HPC) Research Group, Department of Computer Science. His research interests include data engineering, the semantic web, deep learning applications, and high-performance computing.

OSAMA ABULNAJA received the B.S. degree in computer science from King Abdulaziz University, Jeddah, Saudi Arabia, in 1986, and the M.S. and Ph.D. degrees in computer science and engineering from the University of Wisconsin–Milwaukee, Milwaukee, WI, USA, in 1990 and 1996, respectively.

He is a retired Professor of computer science with King Abdulaziz University. His research interests include fault tolerance, high-performance computing, system performance, and systems programming.

TURKI F. AL-SOMANI received the B.Sc. and M.Sc. degrees in electrical and computer engineering from King Abdulaziz University, Saudi Arabia, in 1997 and 2000, respectively, and the Ph.D. degree from the King Fahd University of Petroleum and Minerals (KFUPM), Dhahran, Saudi Arabia, in 2006.

He is currently a Professor with the Cybersecurity Department, Umm Al-Qura University (UQU), Mecca, Saudi Arabia. He holds multiple patents and has authored numerous publications in journals and conference proceedings within these fields. His research interests include information theory, computer arithmetic, and cryptosystems.

ABDULLAH ALTUWAIJRI received the B.S. degree in electrical engineering from King Abdulaziz University (KAU), Jeddah, Saudi Arabia, in 2024, with a specialization in electronics and communications.

He is currently an RTL Engineer with the National Company of Telecommunications and Information Security (NTIS). Additionally, his research focuses on the hardware implementation of accelerators for post-quantum cryptographic algorithms.